The Collaborative Itinerary CRDT

A complete, beginner-to-confident walkthrough of the conflict-free ordered list that powers shared trip planning

Travel Advisor project · fractional-indexing ordering · action-based real-time sync · generated 2026-06-17 · **includes the hardening pass (§12)**

Contents

Read me first. Sections 1–11 teach the core idea with the *original*, single-key design — that's the clearest way to learn it. Section 12 then layers on the **hardening pass** that makes it production-solid: **compound keys** (the order is now `"<frac>#<siteId>"`), a **version counter** for dropped-message detection, an **offline outbox**, and three more fixes. Where §1–11 say the key is just `"m"`, §12 upgrades it to `"m#srv"`.

1 · What problem are we even solving?

The shared itinerary is an **ordered list of stops** that several people edit **at the same time**. Alice drags "Eiffel Tower" to the top while Bob, on his phone, drags "Louvre" to the bottom — both in the same second. The app must end up with **one list that contains both changes**, on every screen, without anyone's edit silently vanishing.

The original bug (why this feature exists). The first version synced the *whole list* with "last write wins": each client sent its entire itinerary, and whichever message arrived last overwrote the other. Two simultaneous reorders → one of them was erased. That is the *exact* failure a collaborative planner cannot have.

We need a scheme where two concurrent reorders of **different items** *merge* automatically rather than fight. That is precisely what a **CRDT** gives us.

2 · CRDT crash course

CRDT = Conflict-free Replicated Data Type. It's a data structure designed so that multiple copies (replicas) can be edited independently and **always merge to the same final value**, regardless of the order updates arrive in. No locks, no "who wins" arbitration.

The magic property is that the merge operation is:

Property	Plain English	Why it matters here
Commutative	A then B = B then A	Network reorders messages; result is identical either way.
Associative	Grouping doesn't matter	Batches of updates merge the same as one-by-one.
Idempotent	Applying twice = applying once	A re-delivered message can't corrupt state.

There are two broad families: **state-based** (send the whole state, merge with a "join" function) and **operation-based** (send small operations that commute). Our itinerary is **operation-based**: we ship tiny `add` / `remove` / `move` actions.

> **Our CRDT is deliberately small.** We do not use a heavyweight library (Yjs, Automerge). The *only* piece that needs CRDT thinking is **ordering**, so we built exactly that one primitive — fractional indexing — and nothing more. Everything else (add/remove) is naturally conflict-free because each item has a unique id.

3 · The big idea: order by a string, not an index

Array indices are the enemy of collaboration. If "position 2" is the source of truth, then the instant someone inserts at position 1, *every* later index shifts — so two people's "move to position 2" mean different things by the time they arrive.

The fix: **each item carries its own `order` string, and the list is simply "items sorted by that string."**

- To **move** an item, we rewrite **only its own** `order` key to a value that sorts between its new neighbours.
- Two people moving two **different** items touch two **different** keys → the edits never collide → they merge.

```
list  =  sortByOrder(items)

  item A   order = "h"
  item B   order = "m"      <-- compare these strings lexicographically
  item C   order = "t"

  move B between A and C  ->  give B a new key strictly between "h" and "t", e.g. "n"
  (nobody else's key changed)
```

So the entire problem reduces to one question:

"Given two keys `a < b`, produce a new key `K` with `a < K < b`."
And you must *always* be able to do it — even when `a` and `b` look adjacent like `"m"` and `"n"`. That's **fractional indexing**.

4 · Fractional indexing from first principles

Think of the keys as **fractions between 0 and 1**. Between any two distinct fractions there's always another one (take the midpoint). 0.5 sits between 0 and 1; 0.25 between 0 and 0.5; 0.375 between 0.25 and 0.5 — you can subdivide forever. Real numbers never "run out of room."

We can't use floating-point (it has finite precision and *does* run out), so we encode the same idea as **strings compared lexicographically**. Here the "digits" are the letters `a` .. `z` (base-26). String comparison works digit-by-digit, exactly like comparing decimals.

Fractions	Our string keys	Idea
0 .. 1	`a` .. `z`	The alphabet is our digit set.
midpoint of 0,1 = 0.5	`keyBetween(null,null)` = `"m"`	Middle digit, maximum elbow room.
between 0.5 and 1	between `"m"` and `null` → a digit above `m`	Open upper bound = "append at end".
0.5 and 0.51 (adjacent)	`"m"` and `"n"` → `"m" + something`	No digit fits between, so **add a digit**: e.g. `"mm"`.

The key insight for adjacent keys. There is nothing between `"m"` and `"n"` at one character. But `"mm"` sorts *after* `"m"` (longer, shares the prefix) and *before* `"n"` (because `m < n` at position 0). So we keep the lower key's digit and **descend into a new character position** — just like writing 0.55 between 0.5 and 0.6. Strings can grow, so we never run out of room.

5 · The load-bearing invariant

Our `keyBetween(a, b)` guarantees three things for valid inputs:

1. **BETWEEN:** `a < K < b` (lexicographically).
2. **NON-EMPTY:** `K.length ≥ 1`.
3. **STABLE TERMINAL:** `K` never ends in the lowest digit `'a'` or the highest digit `'z'`.

Rule (3) is the one that makes the whole thing correct. Because **no key ever ends in `'a'` or `'z'`**:

- You can **always** find a key below any `K`: it doesn't end in `'a'`, so lower the last digit.
- You can **always** find a key above any `K`: it doesn't end in `'z'`, so raise the last digit.
- Therefore every pair of distinct neighbours has a gap, and `keyBetween` is **injective** — it can never map two different neighbour-pairs onto the *same* key.

We treat `'a'` and `'z'` as pure **boundary markers** ("smaller / larger than any real key"). The usable terminal digits are `'b'`..`'y'`.

> **The bug that taught us this.** The very first prototype *passed all the example tests* and still produced **duplicate keys** under random load. It was non-injective: in the adjacent-digit branch it discarded the lower bound's tail, so two different neighbour pairs could map to one key. Two items with the *same* `order` → unstable sort → the list flickers/duplicates. The fix was the "never end in 'a'/'z'" invariant, and the **fuzz test** (§11) exists specifically to catch this class of bug that example tests miss.

6 · The full `fracdex.js`, explained

`server/src/fracdex.js` (the client copy `src/collab/fracdex.js` is identical logic, kept in sync by hand — there's no shared module across the client/server boundary here).

Setup: the digit alphabet

```
const DIGITS = 'abcdefghijklmnopqrstuvwxyz';
const MIN = 0;                 // 'a' — boundary marker, never a terminal digit
const MAX = DIGITS.length - 1; // 'z' (25) — boundary marker, never a terminal digit
const val = (c) => DIGITS.indexOf(c);   // 'c' -> 2
const chr = (v) => DIGITS[v];           // 2   -> 'c'
const midDigit = (lo, hi) => Math.floor((lo + hi) / 2);  // integer midpoint digit
```

The entry point: `keyBetween(lo, hi)`

```
export const keyBetween = (lo, hi) => {
  // Legacy rows predate `order` and carry '' — not a real key, so treat '' as an open end.
  const a = lo || null;
  const b = hi || null;
  if (a !== null && b !== null && a >= b) {
    throw new Error(`keyBetween: lower "${a}" must be < upper "${b}"`);
  }
  if (a === null && b === null) return chr(midDigit(MIN, MAX)); // 'm' — middle, max room
  if (a === null) return before(b);   // open lower bound  -> "insert at front"
  if (b === null) return after(a);    // open upper bound  -> "append at end"
  return between(a, b);               // two real keys     -> squeeze between them
};
```

Four cases: both ends open (the very first item, `"m"`), front insert, end insert, and the general "between two real keys" case. Each delegates to a small helper.

`after(a)` — a key strictly greater than `a`

```
const after = (a) => {
  if (a.length === 0) return chr(midDigit(MIN, MAX));       // 'm'
  const d = val(a[0]);
  if (d <= MAX - 2) {
    // a[0] is 'a'..'x': a usable digit ('b'..'y') sits strictly above it.
    return chr(Math.floor((d + 1 + MAX) / 2));
  }
  if (d === MAX - 1) {
    // a[0] is 'y': nothing usable between 'y' and 'z'. Step to the 'z' boundary
    // (non-terminal) and append a middle digit. "zm..." > any "y..." since 'z' > 'y'.
    return chr(MAX) + chr(midDigit(MIN, MAX));              // -> "zm"
  }
  // a[0] is 'z' (MAX): keep it (it's a non-terminal boundary) and recurse for room above.
  return a[0] + after(a.slice(1));
};
```

`before(b)` — a key strictly less than `b` (mirror image)

```
const before = (b) => {
  if (b.length === 0) return chr(midDigit(MIN, MAX));       // 'm'
  const d = val(b[0]);
  if (d >= MIN + 2) return chr(midDigit(MIN, d));          // usable digit below b[0]
  if (d === MIN + 1) return chr(MIN) + chr(midDigit(MIN, MAX)); // 'b' -> "am"
  return b[0] + before(b.slice(1));                         // 'a' — keep and descend
};
```

`between(a, b)` — the general case

```
const between = (a, b) => {
  let i = 0, prefix = '';
  while (i < a.length && i < b.length && a[i] === b[i]) { prefix += a[i]; i += 1; } // shared prefix
  if (i >= a.length) return prefix + before(b.slice(i));   // a is a prefix of b (e.g. "m","mam")

  const da = val(a[i]);
  const db = val(b[i]);                                     // b is greater, so it has a digit here
  if (db - da > 1) return prefix + chr(midDigit(da, db));  // gap -> a midpoint digit fits
  // Adjacent (db === da+1): no room here. Keep a's digit, then a key strictly ABOVE a's tail
  // (open upper) — stays above a, still below b.
  return prefix + a[i] + after(a.slice(i + 1));
};
```

Two convenience helpers

```
// The key that places a new item AFTER the current last item.
export const keyAfterLast = (lastOrder) => keyBetween(lastOrder || null, null);

// Sort items by their order key. STABLE, so legacy rows sharing an empty key keep order.
export const sortByOrder = (items) =>
  [...items].sort((x, y) => {
    const ox = x.order ?? '', oy = y.order ?? '';
    return ox < oy ? -1 : ox > oy ? 1 : 0;
  });
```

7 · Action-based sync: how edits travel

The client never sends the whole list. It sends one of three tiny **actions**. Each action touches exactly one item, so concurrent actions on *different* items are independent.

Action (emit)	Server broadcasts	What changes
`itinerary:add`	`itinerary:item-added`	One new item, key = `keyAfterLast(last)`.
`itinerary:remove`	`itinerary:item-removed`	One item filtered out by id.
`itinerary:move`	`itinerary:item-moved`	**Only that item's** `order` is rewritten.

> **Server is the source of truth for `order`.** The server broadcasts each change to **everyone, including the original sender**. The sender shows an instant *optimistic* version, then reconciles against the server's authoritative event. This keeps all replicas converging on one set of keys.

8 · The data model & store

`server/src/models/Trip.js` — each itinerary item stores its own `order` string:

```
const itineraryItemSchema = new mongoose.Schema({
  id:        { type: String, required: true },
  placeName: { type: String, default: 'Stop' },
  lat:       { type: Number, required: true },
  lng:       { type: Number, required: true },
  addedBy:   { type: String, default: 'Someone' },
  order:     { type: String, default: '' },   // fractional-index key; list sorts by THIS
}, { _id: false });
```

`server/src/store.js` — the mutators. Note how `move` changes one field only, and how legacy rows self-heal:

```
// Append a new stop after the current last one. Ignores duplicate ids.
export const addItineraryItem = async (code, item) => {
  let added = null;
  const trip = await persist(code, (t) => {
    if (t.itinerary.some((i) => i.id === item.id)) return;
    const last = sortByOrder(t.itinerary).at(-1);
    added = { ...item, order: keyAfterLast(last?.order || null) };
    t.itinerary.push(added);
  });
  return { trip, item: added };
};

// Set ONE item's order key (its new fractional-index position). Only that item changes.
export const moveItineraryItem = async (code, itemId, order) => {
  let moved = null;
  const trip = await persist(code, (t) => {
    const it = t.itinerary.find((i) => i.id === itemId);
    if (!it) return;
    it.order = order;
    moved = { id: it.id, order };
  });
  return { trip, item: moved };
};
```

Self-healing legacy data (`backfillOrder`)

```
// Ensure every item has a fractional-index `order`, preserving current array order.
// Trips created before `order` existed have order '' — this gives them real keys so the
// sort and keyBetween() never see an invalid empty bound. Runs on load (getTrip).
const backfillOrder = (trip) => {
  if (!trip?.itinerary?.length) return false;
  let prev = null, changed = false;
  for (const item of trip.itinerary) {
    if (!item.order) { item.order = keyAfterLast(prev); changed = true; }
    prev = item.order;
  }
  return changed;
};
```

9 · The real-time layer

`server/src/realtime.js` — each trip is a Socket.IO **room** keyed by the trip code; broadcasts are scoped to that room so edits never leak across trips.

```
socket.on('itinerary:move', async ({ itemId, order }) => {
  if (!joinedCode) return;
  const { trip, item: moved } = await moveItineraryItem(joinedCode, itemId, order);
  if (trip && moved) io.to(joinedCode).emit('itinerary:item-moved', moved);
});
```

On join, the server sends the full state once, already sorted:

```
socket.emit('trip:state', {
  code: trip.code,
  pins: trip.pins,
  itinerary: sortByOrder(trip.itinerary),   // canonical order from the keys
  you: { name: me.name, color: me.color },
});
```

10 · The client hook: optimistic + reconcile

`src/collab/useCollabTrip.js` — the move action computes a key locally, updates the UI immediately, then emits:

```
const moveItineraryItem = useCallback((itemId, targetIndex) => {
  const sorted  = sortByOrder(itineraryRef.current);
  const without = sorted.filter((i) => i.id !== itemId);
  const clamped = Math.max(0, Math.min(targetIndex, without.length));
  const before  = without[clamped - 1]?.order ?? null;   // new left neighbour
  const after   = without[clamped]?.order ?? null;       // new right neighbour
  let order;
  try { order = keyBetween(before, after); }
  catch { return; }   // neighbours not strictly ordered (shouldn't happen) — skip, don't crash
  setItinerary((prev) =>
    sortByOrder(prev.map((i) => (i.id === itemId ? { ...i, order } : i)))
  );
  socketRef.current?.emit('itinerary:move', { itemId, order });
}, []);
```

And the listeners reconcile incoming server events (the server's key wins, then re-sort):

```
socket.on('itinerary:item-moved', ({ id, order }) =>
  setItinerary((prev) =>
    sortByOrder(prev.map((i) => (i.id === id ? { ...i, order } : i)))
  )
);
```

> **Optimize Route reuses the same conflict-free path.** When the TSP optimizer reorders the whole list, the hook mints a fresh chain of strictly-increasing keys (`keyAfterLast` repeatedly) and emits one `move` per stop — so a bulk reorder flows through the exact same merge machinery as a single manual drag.

11 · The fuzz test — why 300k random checks

`server/test/fracdex.fuzz.mjs` (`npm test` in `server/`). Example tests are **not enough** for a hand-rolled CRDT primitive — the first prototype passed examples and still broke. This is property-based testing: assert the *invariants* hold across tens of thousands of random operations.

#	Scenario	What it proves
1–2	Right/left-edge squeeze (5k each)	Repeated insert next to a bound stays strictly between; keys don't blow up.
3–4	Repeated front / end insert (5k each)	Open-bound paths (`before` / `after`) stay correct under churn.
5	Insert churn (20k)	Maintaining a real sorted list yields strictly increasing, unique keys.
6	Move churn (20k)	The core CRDT op — random reorders never collide.
7	Concurrent merge	Two people moving two different items from one snapshot → deterministic `ZYX` .
8	Injectivity (40k inserts)	**The exact property the first prototype violated**: 40k inserts → 40k unique keys, max length ~9.

```
const endsBad = (k) => k.length > 0 && (k[k.length-1] === 'a' || k[k.length-1] === 'z');
const checkKey = (a, b, k) => {
  assert(k.length >= 1,            `(2) empty key for (${a},${b})`);
  assert(a === null || a < k,      `(1) lower: ${a} !< ${k}`);
  assert(b === null || k < b,      `(1) upper: ${k} !< ${b}`);
  assert(!endsBad(k),              `(3) terminal 'a'/'z': ${k}`);
};
```

Test 8 maintains a growing `Set` of every key handed out and asserts each new key is unseen — directly catching the duplicate-key (non-injective) class of bug.

12 · The hardening pass: 7 fixes

The core in §1–11 is correct, but a self-review found seven gaps between "correct in theory" and "solid in production." All seven were fixed. The unifying theme: the ordering *math* was never the weak point — the **sync protocol and edge cases around it** were. Here's each one.

Fix 1 + 2 — Compound keys (the only real divergence hole)

`keyBetween` is injective for **one** caller. But two people editing from the *same snapshot* can independently compute the **same** fractional key — e.g. both pick `"m"` between `"h"` and `"t"`. Two items with equal keys → their relative order is undefined, and with no tiebreaker two screens can sort them *differently and never converge*. That is the one thing a CRDT must never allow.

> **The fix:** the order is now a **compound key** `"<frac>#<siteId>"`. We compare by the fractional part first, then by a per-client `siteId` — a deterministic, replica-independent tiebreak. Same frac on two sites → every replica still agrees on the final order. The server stamps `srv`; each client gets a random site id.

```
const SEP = '#';
export const stripSite  = (order) => (order == null ? '' : String(order).split(SEP)[0]);
const        siteOf     = (order) => { const s = String(order ?? ''); const i = s.indexOf(SEP);
                                       return i === -1 ? '' : s.slice(i + 1); };
export const makeOrder  = (frac, siteId = '') => (siteId ? `${frac}${SEP}${siteId}` : frac);

// Total order: fractional part first, then site id. (keyBetween strips the site for its math.)
export const compareOrder = (a, b) => {
  const fa = stripSite(a), fb = stripSite(b);
  if (fa !== fb) return fa < fb ? -1 : 1;
  const sa = siteOf(a),    sb = siteOf(b);
  return sa < sb ? -1 : sa > sb ? 1 : 0;
};

// sortByOrder now: compareOrder first, then a final fallback on `id` (stable even for legacy
// rows that share an empty key and no site).
export const sortByOrder = (items) =>
  [...items].sort((x, y) => {
    const c = compareOrder(x.order ?? '', y.order ?? '');
    if (c !== 0) return c;
    const ix = x.id ?? '', iy = y.id ?? '';
    return ix < iy ? -1 : ix > iy ? 1 : 0;
  });
```

The fuzz test gained scenarios (§9–10) that mint the *same* frac on two sites and assert `compareOrder` stays a strict total order and both replicas sort identically.

Fix 3 — Optimistic edits are marked "unconfirmed"

When you add or move a stop, the UI applies it instantly with a *provisional* key, then the server broadcasts the authoritative one. Between those, the item now carries `unconfirmed: true`; the panel **dims it and**

shows "syncing…" until the server's event clears the flag. No more trusting a provisional key as if it were final.

Fix 4 + 5 — Version counter, gap-detection, resync & offline outbox

Each trip now has a monotonic `version`, bumped on **every real mutation** (no-ops like a duplicate add don't bump it). Every broadcast carries the new version, so a client can detect a **dropped message** (a gap in the sequence) and ask for a full resync.

```
// CLIENT — applied to every broadcast. Ignore stale events; resync on a gap.
const applyVersion = (v) => {
  if (typeof v !== 'number') return true;          // versionless correction → always apply
  if (v <= versionRef.current) return false;        // stale / duplicate → ignore
  if (v > versionRef.current + 1 && versionRef.current !== -1) {
    socket.emit('trip:resync');                     // missed an event → get authoritative state
  }
  versionRef.current = v;
  return true;
};
```

And actions emitted while the socket is **disconnected** are queued in an **outbox** and replayed in order on reconnect (after a fresh `trip:join` re-bases the version) — so a brief drop (the "phone in a tunnel" case) doesn't silently lose your edits.

```
// Emit now if connected, else queue for replay on reconnect.
const sendOrQueue = useCallback((event, payload) => {
  const socket = socketRef.current;
  if (socket && socket.connected) socket.emit(event, payload);
  else outboxRef.current.push({ event, payload });
}, []);
```

> Replaying a queued action after a resync is **safe by construction**: an `add` dedupes by id, a `move` just re-sets one key, and a `remove` of an already-gone item is a no-op. This is exactly the idempotence property from §2 paying off.

Fix 6 — The two fracdex copies can't silently drift

`server/src/fracdex.js` and `src/collab/fracdex.js` are hand-duplicated (no shared module across the client/server split). A new **parity test** imports *both* and asserts they produce byte-identical results — success values *and* thrown-error behaviour — across ~26k cases. It runs as part of `npm test`, so editing one copy without mirroring the other fails CI.

```
import * as server from '../src/fracdex.js';
import * as client from '../../src/collab/fracdex.js';
// ...assert server.keyBetween(lo,hi) === client.keyBetween(lo,hi) across a 20k-op shared vector,
//    plus keyAfterLast, makeOrder, compareOrder, sortByOrder.
```

Fix 7 — The move/remove race no longer leaves a ghost

If Alice moves item X while Bob removes X, Alice's `move` arrives at a deleted item. The server already didn't crash (`if (!it) return`) — but Alice's optimistically-moved X used to **linger on her screen** with nothing to remove it. Now the move reports `orphaned: true`, and the realtime layer sends a `item-removed` back to just the mover (versionless, so it always applies):

```
socket.on('itinerary:move', async ({ itemId, order }) => {
  const { trip, item: moved, orphaned } = await moveItineraryItem(joinedCode, itemId, order);
  if (trip && moved)  io.to(joinedCode).emit('itinerary:item-moved', { ...moved, version: trip.version });
  else if (orphaned)  socket.emit('itinerary:item-removed', { itemId }); // drop the ghost
});
```

#	Gap	Fix in one line
1	Same-slot concurrent move → duplicate key	Compound key `frac#site`.
2	No tiebreak → replicas diverge	`compareOrder` = total order (frac → site → id).
3	Provisional key trusted as final	`unconfirmed` flag + dimmed "syncing…".
4	Dropped message → silent divergence	Per-trip `version` + gap detection + `trip:resync`.
5	Reconnect loses offline edits	Outbox queue replayed on reconnect.
6	Two fracdex copies drift	Parity test (26k checks) in `npm test`.
7	move/remove race leaves a ghost	`orphaned` → remove-back to the mover.

Verified: `npm test` → fuzz 309,059 checks + parity 26,262 checks, all green; `react-scripts build` compiles clean.

13 · Worked example: two people, one merge

Start with three stops X, Y, Z (keys minted left to right):

```
X = "m"        (keyBetween(null,null))
Y = after(X)   -> "t"      (keyAfterLast "m")
Z = after(Y)   -> "w"      (keyAfterLast "t")
list = X Y Z
```

From that same snapshot, simultaneously:

```
Priya moves Z to the FRONT:  keyBetween(null, X="m") -> before("m") = "f"   so Z.order = "f"
Aman  moves X to the END:    keyAfterLast(Z="w")     -> after("w")  = "x"   so X.order = "x"
```

Both `move` actions reach the server (in any order) and are broadcast. Final keys:

```
Z = "f"
Y = "t"
X = "x"
sortByOrder -> f < t < x  ->  Z, Y, X
```

Both edits survived. Priya touched only Z's key, Aman only X's. No overwrite, no lost update — the list deterministically becomes `Z Y X` on every screen. That is the entire promise of the CRDT, demonstrated. (This is literally fuzz test §11 scenario 7, which asserts the result equals `"ZYX"` .)

14 · Glossary & cheat-sheet

Term	Meaning in this project
CRDT	Conflict-free Replicated Data Type — edits merge deterministically with no coordination.
Fractional index	The `<frac>` part of the order; between any two there's always another, so you can insert/move forever.
Compound key	The stored order: `"<frac>#<siteId>"`. Compared frac-then-site so two replicas with the same frac still agree (fix 1+2).
siteId	A per-client tag stamped onto keys it mints; the deterministic tiebreaker when two clients pick the same frac.
keyBetween(a,b)	The one primitive: a frac strictly between `a` and `b` (strips any site first; either bound may be `null` = open end).
compareOrder	Total order over compound keys: frac, then site. Backed by an `id` fallback in `sortByOrder`.
Invariant (3)	"A key never ends in 'a' or 'z'." Guarantees room above/below every key → injectivity.
Injective	Different neighbour-pairs never map to the same frac → no duplicate keys from a single caller.
Action-based sync	Ship tiny add/remove/move ops, not the whole list → concurrent ops on different items merge.
version	Per-trip monotonic counter bumped on each real mutation; a gap tells a client it missed an event (fix 4).
trip:resync	Client request for full authoritative state after it detects a version gap.
Outbox	Queue of actions emitted while offline, replayed in order on reconnect (fix 5).
unconfirmed	Flag on an optimistic item until the server confirms it; UI dims it as "syncing…" (fix 3).
orphaned move	A move targeting an already-removed item; server tells the mover to drop the ghost (fix 7).
Optimistic update	Apply the change locally for instant feedback, then reconcile with the server's broadcast.
Source of truth	The server owns the canonical `order` + `version`; clients reconcile their optimistic state to it.
backfillOrder	Self-heals legacy rows (empty `order`) on load so the primitive never sees a bad bound.
Parity test	Asserts the client & server fracdex copies behave byte-identically; runs in `npm test` (fix 6).

One-sentence summary. Each stop carries a sortable compound `order` key `"<frac>#<site>"`; moving a stop rewrites only that one key to sit between its new neighbours; because `keyBetween` always finds a strictly-between key and the site id breaks any frac tie (a total order, proven by a 300k-check fuzz test + a client/server parity test), and because every change is a tiny versioned action with offline replay and resync-on-gap, concurrent reorders by different people merge instead of clobbering and survive flaky connections — a small but solid, purpose-built CRDT.

Files referenced: `server/src/fracdex.js`, `src/collab/fracdex.js`, `server/test/fracdex.fuzz.mjs`, `server/test/fracdex.parity.mjs`, `server/src/store.js`, `server/src/realtime.js`, `server/src/models/Trip.js`, `src/collab/useCollabTrip.js`, `src/components/Collab/CollabPanel.jsx`.

All code excerpts are verbatim from the project at generation time, including the §12 hardening pass.